

RIPHAH INTERNATIONAL COLLEGE HARBANSPURA CAMPUS

A Project of Riphah International University

ADP (ITM) — Artificial Intelligence

Industry-Ready Student Project Catalog

Part 2 of 2 | Projects 11–20

Semester: 3rd | Course: Introduction to Artificial Intelligence

Program: ADP (ITM) | Credit Hours: 3

Introduction — Part 2 of 2

This document is the second and final part of the ADP (ITM) AI Project Catalog, covering Projects 11 through 20. Together with Part 1 (Projects 1–10), this catalog provides a complete set of 20 industry-ready project ideas for students enrolled in the Introduction to Artificial Intelligence course at Riphah International Colleges.

Each project in this part covers a distinct area of Artificial Intelligence including NLP, Regression, Image Processing, Computer Vision, Expert Systems, Recommendation Systems, Constraint Satisfaction, and more. All projects are scoped for a 6–8 week development window and aligned with the course CLOs.

Part 2 — Projects at a Glance

#	Project Title	Category
11	AI-Powered Resume Screening System	Machine Learning / NLP
12	Smart Traffic Flow Prediction System	Machine Learning / Regression
13	AI-Based Crop Disease Detection from Leaf Images	Machine Learning / Image Processing
14	Intelligent Inventory Demand Forecasting System	Machine Learning / Regression / Time Series
15	Fake News Detection System using NLP	Machine Learning / NLP
16	AI Personal Finance Advisor Chatbot	Expert Systems / Knowledge Representation
17	Movie Recommendation System using Collaborative Filtering	Machine Learning / Unsupervised / Recommendation
18	AI-Powered Object Detection in Images	Machine Learning / Computer Vision
19	AI Sudoku Solver using Constraint Satisfaction	AI Problem Solving / Search Algorithms
20	AI-Based Text Summarization Tool	Natural Language Processing / Machine Learning

Submission Reminder: Each student or group must confirm their project selection with the course instructor before beginning development. Duplicate selections within the same section are not permitted.

Project 11: AI-Powered Resume Screening System

Category	Development Timeline	Difficulty Level
Machine Learning / NLP	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project builds an automated resume screening tool that uses NLP and supervised machine learning to match candidate resumes against a given job description and rank them by relevance. Students will preprocess raw resume text, apply TF-IDF vectorization, compute cosine similarity scores, and optionally train a classifier to label resumes as Shortlisted or Rejected. The system mimics a real-world HR automation workflow used in industry. This project aligns with course units on Natural Language Processing, Tokenization, Stemming, Stop Words removal, Bag of Words, TF-IDF, and Supervised Learning.

Detailed Functional Requirements

1. The system shall allow HR users to paste or upload a job description text as the matching baseline.
2. The system shall accept multiple resume documents (plain text or PDF) for batch processing.
3. The system shall preprocess all text by performing tokenization, stop-word removal, and stemming.
4. The system shall apply TF-IDF vectorization and compute cosine similarity between each resume and the job description.
5. The system shall rank all uploaded resumes from highest to lowest match score.
6. The system shall classify each resume as Shortlisted (score above threshold) or Rejected automatically.
7. The system shall display a ranked results table showing candidate name, match score, and status.
8. The system shall highlight key matching keywords found in each resume relative to the job description.
9. The system shall allow the HR user to adjust the shortlisting threshold percentage from the interface.
10. The system shall export the ranked results list as a downloadable CSV report.

Tools and Technologies

Python 3.x, NLTK, Scikit-learn (TF-IDF, Cosine Similarity), PyPDF2 or pdfplumber, Pandas, Streamlit or Flask, PyCharm

Project 12: Smart Traffic Flow Prediction System

Category	Development Timeline	Difficulty Level
Machine Learning / Regression	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project develops a machine learning model that predicts traffic density or vehicle count on a road segment based on time-of-day, day-of-week, weather conditions, and historical traffic data. Students will apply regression techniques and classification models such as KNN to categorize traffic levels as Low, Moderate, or High. The system will include a visualization dashboard displaying predicted traffic trends over a 24-hour period. This project aligns with course units on Supervised Learning, Regression vs Classification, KNN, Dataset Preparation, and Data Visualization.

Detailed Functional Requirements

1. The system shall accept a historical traffic dataset in CSV format containing time, weather, and vehicle count columns.
2. The system shall preprocess data by encoding categorical features, handling missing values, and normalizing numerical fields.
3. The system shall train a KNN classifier to categorize traffic as Low, Moderate, or High density.
4. The system shall implement a regression model to predict estimated vehicle count for a given time and conditions.
5. The system shall split data into training (70%), validation (15%), and test (15%) sets and report accuracy.
6. The system shall allow users to input parameters (time, day, weather) and receive a real-time traffic prediction.
7. The system shall display a 24-hour traffic trend line chart for a selected day and weather condition.
8. The system shall generate a model evaluation report with accuracy, MAE, and RMSE metrics.
9. The system shall visualize feature importance to show which factors most affect traffic density.
10. The system shall log all user prediction queries with timestamps to a local history file.

Tools and Technologies

Python 3.x, Scikit-learn (KNN, LinearRegression), Pandas, NumPy, Matplotlib, Seaborn, Streamlit, PyCharm

Project 13: AI-Based Crop Disease Detection from Leaf Images

Category	Development Timeline	Difficulty Level
Machine Learning / Image Processing	7–8 Weeks	Intermediate

CLO Coverage: CLO4, CLO5

Abstract

This project develops an image classification system that identifies plant leaf diseases from uploaded photographs. Students will use a pre-trained deep learning model (transfer learning) or a trained ANN to classify leaf images into categories such as Healthy, Bacterial Blight, or Leaf Rust. OpenCV will be used for image preprocessing including resizing, normalization, and color space conversion. The project aligns with course units on Image Processing, Artificial Neural Networks, Deep Learning Introduction, and Machine Learning Classification.

Detailed Functional Requirements

1. The system shall accept leaf images in JPG or PNG format as input through a file upload interface.
2. The system shall preprocess uploaded images by resizing to 224x224 pixels, converting to RGB, and normalizing pixel values.
3. The system shall apply a trained ANN or pre-trained CNN model to classify the leaf into one of at least 5 disease categories.
4. The system shall display the predicted disease class and a confidence score for the prediction.
5. The system shall display the top-3 possible disease classifications with their confidence percentages.
6. The system shall show the uploaded image alongside its preprocessed version for user comparison.
7. The system shall provide a brief description of each detected disease and recommended treatment action.
8. The system shall support batch image upload and display results for each image in a results table.

9. The system shall visualize model accuracy and training loss curves from the training process.
10. The system shall export prediction results including image name, predicted class, and confidence to a CSV report.

Tools and Technologies

Python 3.x, OpenCV, TensorFlow / Keras or Scikit-learn, NumPy, Matplotlib, Streamlit or Tkinter, PlantVillage Dataset, PyCharm

Project 14: Intelligent Inventory Demand Forecasting System

Category	Development Timeline	Difficulty Level
Machine Learning / Regression / Time Series	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project builds a machine learning-based demand forecasting system for retail inventory management. Students will process historical sales data, engineer time-based features such as month, season, and day-of-week, and train regression models to predict future product demand. The system will help businesses avoid stockouts and overstock situations. This project maps to course units on Supervised Learning, Regression, Dataset Preparation, and Data Visualization with Matplotlib.

Detailed Functional Requirements

1. The system shall accept historical sales data in CSV format containing product ID, date, and units sold columns.
2. The system shall perform feature engineering to extract month, quarter, day-of-week, and seasonal indicators from date fields.
3. The system shall train a regression model (Linear Regression and KNN Regressor) to predict future demand values.
4. The system shall evaluate both models using MAE, RMSE, and R-squared metrics on a held-out test set.
5. The system shall allow users to select a product and forecast demand for a user-specified future period (days/weeks).

6. The system shall display a time-series line chart comparing actual historical sales versus predicted demand.
7. The system shall generate an alert when predicted demand exceeds current stock levels by a set threshold.
8. The system shall support multi-product forecasting by allowing selection from a dropdown list of product IDs.
9. The system shall display a feature importance chart showing which time features most influence demand.
10. The system shall export the forecast results with predicted dates and demand values to a CSV file.

Tools and Technologies

Python 3.x, Scikit-learn (LinearRegression, KNeighborsRegressor), Pandas, NumPy, Matplotlib, Seaborn, Streamlit or Flask, PyCharm

Project 15: Fake News Detection System using NLP

Category	Development Timeline	Difficulty Level
Machine Learning / NLP	6–8 Weeks	Intermediate

CLO Coverage: CLO2, CLO4, CLO5

Abstract

This project builds a binary text classification system that detects whether a given news article or headline is Real or Fake using NLP preprocessing and supervised machine learning classifiers. Students will process a labeled news dataset, apply Bag-of-Words and TF-IDF feature extraction, train Naive Bayes and KNN models, and evaluate their accuracy. A web interface will allow users to paste news text and receive an instant authenticity prediction. This project aligns with course topics on NLP, Tokenization, Bag of Words, Stop Words, Naive Bayes, KNN, and Supervised Learning.

Detailed Functional Requirements

1. The system shall accept a labeled fake/real news CSV dataset for model training.
2. The system shall preprocess article text through tokenization, stop-word removal, and stemming using NLTK.
3. The system shall apply both Bag-of-Words and TF-IDF vectorization and compare their effect on model accuracy.

4. The system shall train Naive Bayes and KNN classifiers and report individual accuracy, precision, recall, and F1-score.
5. The system shall allow users to paste custom news text and classify it as Real or Fake in real time.
6. The system shall display the prediction confidence percentage alongside the classification label.
7. The system shall highlight the top keywords that most influenced the classification decision.
8. The system shall generate and display a confusion matrix for each trained classifier.
9. The system shall allow side-by-side comparison of Naive Bayes vs KNN results on the same input.
10. The system shall log all user classification queries with predictions and confidence scores to a history file.

Tools and Technologies

Python 3.x, NLTK, Scikit-learn (Naive Bayes, KNN, TF-IDF), Pandas, Matplotlib, Streamlit or Flask, PyCharm

Project 16: AI Personal Finance Advisor Chatbot

Category	Development Timeline	Difficulty Level
Expert Systems / Knowledge Representation	6–7 Weeks	Intermediate

CLO Coverage: CLO3, CLO4, CLO5

Abstract

This project develops a rule-based intelligent personal finance advisor chatbot that uses a knowledge base of financial rules and forward chaining inference to provide personalized budgeting advice, savings recommendations, and expense alerts. Students will design the knowledge base, implement the inference engine in Python, and build a conversational interface. The system will cover expert system concepts including knowledge base design, inference strategies, and user session management. This project aligns with course units on Expert Systems, Knowledge Base, Inference Engine, Forward Chaining, and Backward Chaining.

Detailed Functional Requirements

1. The system shall accept user financial data inputs including monthly income, fixed expenses, and variable spending categories.

2. The system shall implement a knowledge base of at least 40 financial rules covering budgeting, savings, and debt management.
3. The system shall apply forward chaining to derive financial recommendations from user-provided facts.
4. The system shall classify user financial health as Healthy, Moderate Risk, or Critical based on income-expense ratio rules.
5. The system shall recommend a monthly savings target based on the 50-30-20 budgeting rule logic.
6. The system shall generate spending alerts when any expense category exceeds a rule-defined threshold.
7. The system shall present the full inference chain showing which rules were fired to produce each recommendation.
8. The system shall support multi-turn conversation allowing the user to update financial data and receive revised advice.
9. The system shall generate a monthly financial summary report with total income, expenses, savings, and recommendations.
10. The system shall allow the administrator to add, edit, or remove rules from the knowledge base through a configuration file.

Tools and Technologies

Python 3.x, JSON or SQLite (Knowledge Base), Tkinter or Streamlit (Chat UI), Pandas, Matplotlib (charts), PyCharm

Project 17: Movie Recommendation System using Collaborative Filtering

Category			Development Timeline	Difficulty Level
Machine Learning / Unsupervised Recommendation		/	6–8 Weeks	Intermediate

CLO Coverage: CLO4, CLO5

Abstract

This project builds a movie recommendation engine using collaborative filtering and content-based filtering techniques. Students will work with the MovieLens dataset, compute user-item similarity matrices using cosine similarity, and generate personalized movie recommendations for a selected user. A simple web dashboard will display recommended movies with predicted

ratings. This project aligns with course topics on Machine Learning, Dimensionality Reduction (PCA), Unsupervised Learning principles, and data visualization with Matplotlib and Seaborn.

Detailed Functional Requirements

1. The system shall load and preprocess the MovieLens ratings dataset including user ID, movie ID, and rating columns.
2. The system shall construct a user-item rating matrix from the dataset.
3. The system shall implement user-based collaborative filtering using cosine similarity to find similar users.
4. The system shall implement item-based collaborative filtering as a secondary recommendation approach.
5. The system shall generate a top-10 movie recommendation list for any selected user ID.
6. The system shall display each recommendation with predicted rating and genre information.
7. The system shall apply content-based filtering using movie genre features as an alternative recommendation mode.
8. The system shall allow users to rate a set of seed movies and receive personalized recommendations based on those inputs.
9. The system shall visualize the distribution of ratings across all movies using a histogram chart.
10. The system shall display a heatmap of the similarity matrix for a subset of users or movies.

Tools and Technologies

Python 3.x, Scikit-learn (Cosine Similarity, PCA), Pandas, NumPy, Matplotlib, Seaborn, Streamlit or Jupyter Notebook, MovieLens Dataset, PyCharm

Project 18: AI-Powered Object Detection in Images

Category	Development Timeline	Difficulty Level
Machine Learning / Computer Vision	7–8 Weeks	Intermediate

CLO Coverage: CLO1, CLO4, CLO5

Abstract

This project implements an object detection system that identifies and labels multiple objects within uploaded images using pre-trained models such as YOLO or a simpler OpenCV-based

DNN classifier. Students will learn image preprocessing, bounding box drawing, class label mapping, and confidence thresholding. The system provides a visual output with detected objects highlighted with labeled bounding boxes. This project aligns with course units on Image Processing, Introduction to Deep Learning, Artificial Neural Networks, and Generative AI applications overview.

Detailed Functional Requirements

1. The system shall accept image files (JPG, PNG) as input through a file upload or drag-and-drop interface.
2. The system shall load and apply a pre-trained object detection model (YOLO or OpenCV DNN) for inference.
3. The system shall preprocess input images including resizing, mean subtraction, and blob creation for the model.
4. The system shall detect all objects in the image and draw labeled bounding boxes around each detected object.
5. The system shall display the class label and confidence score for every detected object on the output image.
6. The system shall apply Non-Maximum Suppression to eliminate overlapping redundant bounding boxes.
7. The system shall allow the user to set a minimum confidence threshold to filter low-confidence detections.
8. The system shall display a summary table listing all detected object classes and their respective counts.
9. The system shall support processing multiple images in a batch and display all results in a gallery view.
10. The system shall save the annotated output image with bounding boxes to disk for user download.

Tools and Technologies

Python 3.x, OpenCV (DNN module), YOLOv3 or YOLOv5 weights, NumPy, Matplotlib, Streamlit or Tkinter, COCO Dataset labels, PyCharm

Project 19: AI Sudoku Solver using Constraint Satisfaction

Category	Development Timeline	Difficulty Level
AI Problem Solving / Search Algorithms	5–6 Weeks	Intermediate

CLO Coverage: CLO2, CLO3

Abstract

This project implements an intelligent Sudoku puzzle solver using Constraint Satisfaction Problem (CSP) techniques combined with backtracking search and arc consistency algorithms. Students will model Sudoku as a CSP with variables, domains, and constraints, then apply systematic backtracking with forward checking to solve any valid Sudoku puzzle. A graphical interface will display the puzzle and animate the solving process step by step. This project aligns with course units on Problem Solving Agents, Search Strategies, Uninformed and Informed Search, and Problem Formulation.

Detailed Functional Requirements

1. The system shall provide a 9x9 Sudoku grid interface where users can input an unsolved puzzle.
2. The system shall model the Sudoku puzzle as a Constraint Satisfaction Problem with defined variables, domains, and constraints.
3. The system shall implement a backtracking search algorithm to solve the puzzle systematically.
4. The system shall apply forward checking to reduce variable domains and prune invalid candidates during search.
5. The system shall apply the Minimum Remaining Values (MRV) heuristic to select the next cell to fill.
6. The system shall animate the solving process step by step, highlighting the current cell being evaluated.
7. The system shall detect and report if a puzzle has no valid solution due to invalid initial values.
8. The system shall display the number of backtracks performed and the total time taken to solve the puzzle.
9. The system shall include a puzzle generator that creates valid Sudoku puzzles at Easy, Medium, and Hard levels.
10. The system shall allow users to pause and resume the animated solving process at any step.

Tools and Technologies

Python 3.x, Tkinter or Pygame (GUI), NumPy, Collections, PyCharm / VS Code

Project 20: AI-Based Text Summarization Tool

Category	Development Timeline	Difficulty Level
----------	----------------------	------------------

Natural Language Processing / Machine Learning	6–7 Weeks	Intermediate
--	-----------	--------------

CLO Coverage: CLO2, CLO5

Abstract

This project builds a text summarization application that condenses long articles or documents into concise summaries using both extractive and abstractive NLP techniques. Students will implement extractive summarization using TF-IDF sentence scoring and frequency-based ranking, and explore abstractive summarization using a pre-trained transformer model. The tool will be packaged as a web application where users paste or upload text and receive a generated summary. This project aligns with course units on NLP, Tokenization, Stemming, Stop Words, Bag of Words, TF-IDF, and Introduction to Generative AI and LLMs.

Detailed Functional Requirements

1. The system shall accept long-form text input via a text area or uploaded plain text file.
2. The system shall implement extractive summarization by scoring sentences using TF-IDF term frequency weighting.
3. The system shall rank all sentences by score and extract the top N sentences as the extractive summary.
4. The system shall allow users to control the summary length by specifying the desired number of sentences or percentage.
5. The system shall implement a word frequency-based alternative extractive method for comparison.
6. The system shall display the extractive summary with key sentences highlighted in the original text.
7. The system shall integrate a pre-trained abstractive summarization model (HuggingFace pipeline) as an advanced option.
8. The system shall display both extractive and abstractive summaries side by side for the same input text.
9. The system shall calculate and display compression ratio (original word count vs summary word count).
10. The system shall allow users to copy the generated summary to clipboard or download it as a plain text file.

Tools and Technologies

Python 3.x, NLTK, Scikit-learn (TF-IDF), HuggingFace Transformers (optional), Pandas, Streamlit or Flask, PyCharm